

GPSDO v1.06 — El manual completo (de cero a LOCK)

English | Polski | Español

Firmware: GPSDO v1.06-rtos de jmnlabs (sobre el GPSDO original de André Balsa, el lazo PI de Lars Walenius y el acumulador multinivel de Alan Cashin) **Placa:** WeAct BlackPill STM32F411CE · **OCXO:** 10 MHz (p. ej. Vectron C4550) **GPS:** receptor u-blox en Serial1 (LEA-M8T / NEO-M8T / ZED-F9T o NEO-6M/7M)

Autor: Jarosław Marek Niewiński (jmnlabs) — **Asistente:** GLM-5.3 Max (Z.ai), incluidas las versiones polaca y española de este manual Este manual no presupone **nada**. Si nunca ha compilado firmware, nunca ha grabado un microcontrolador y nunca ha usado un terminal serie, empiece en la Parte 1 y haga exactamente lo que ahí se dice, en orden. Cada paso indica qué escribir y qué debería ver. Nada importante queda como «ejercicio para el lector».

Resumen en 60 segundos: instale Arduino IDE con el núcleo STM32 (no el 3.0.0), abra el sketch, elija su pantalla en `gpsdo_config.h`, compile con *Newlib Nano + Float printf*, grabe con ST-Link o DFU. **Nunca haga un borrado completo del chip (Erase Chip)** — sus ajustes y calibración viven en el sector 7 y solo un borrado completo puede destruirlos. Después, por serie a 115200: CT, espere, LC, espere, LA 12, SAW 1, ES. Mire cómo la fase se mantiene en cero. Listo.

Índice

- Parte 1 — Compilación del firmware
- Parte 2 — Grabar el firmware, y la regla del sector 7
- Parte 3 — Primer arranque: calibración, algoritmo, guardado
- Parte 4 — Los trece algoritmos (0–12) y sus parámetros
- Parte 5 — Pantallas: qué significa cada campo
- Parte 6 — Telemetría serie (el informe de 6 líneas)
- Parte 7 — Referencia de comandos (todos)
- Parte 8 — El sintonizador en el PC
- Parte 9 — Ajustes, el flash ring y el desgaste
- Parte 10 — Resolución de problemas
- Apéndice A — Cómo funciona un GPSDO, en palabras llanas
- Apéndice B — PID para reacios

Parte 1 — Compilación del firmware

1.1 Qué hay que instalar

- 1. **Arduino IDE** (1.8.x o 2.x — ambos sirven).
- 2. El núcleo **STM32duino**, versión **2.2.0 a 2.12.0**. Boards Manager → busque «stm32» → **STM32 MCU based boards by STMicroelectronics**. Instale la 2.12.0 (la última soportada).

AVISO — el núcleo 3.0.0 no funciona. Salió en julio de 2026 y rompe este proyecto por tres vías: `1toa()` ya no existe (error de compilación), `HardwareSerial Serial12(PA3, PA2)` no compila, y `TFT_eSPI` deja de manejar el panel (pantalla blanca permanente). Quédese en la 2.12.0 o anterior hasta que el proyecto anuncie lo contrario.

1. **Librerías** (Library Manager, instale por nombre exacto):

Librería	Para qué
STM32duino FreeRTOS	el sistema operativo
TinyGPS++	análisis de frases GPS
U8g2	pantallas OLED (solo si activa una)
Adafruit AHTX0	sensor de temperatura/humedad AHT10/AHT20
Adafruit BMP280	sensor de presión/temperatura BMP280
Adafruit INA219	monitor de tensión/corriente de alimentación
hd44780	LCD de caracteres 20x4 (solo si lo activa)
TFT_eSPI	pantalla TFT por SPI

1. Nada más. No hay librería EEPROM — los ajustes viven en el flash del propio chip (Parte 2).

1.2 Abrir el sketch

Descomprima el proyecto. Abra `GPSDO_FreeRTOS/GPSDO_FreeRTOS.ino` en Arduino IDE. La carpeta debe conservar su nombre — Arduino exige que el nombre de la carpeta coincida con el del archivo `.ino`.

1.3 Elegir el hardware en `gpsdo_config.h`

Este único archivo decide qué se compila. Las líneas activas empiezan por `#define`; las apagadas, por `//`. Descomente exactamente lo que tiene su placa y comente lo que no. La configuración de fábrica es una placa completa (TFT grande + todos los sensores); adapte la a la realidad.

Pantalla principal — elija exactamente una:

Marca	Hardware
GPSDO_TFT_ILI9488	TFT SPI 320x480 (por defecto; trabaja en horizontal 480x320)
GPSDO_TFT_ST7789	TFT SPI 240x320
GPSDO_TFT_ILI9341	TFT SPI 240x320
GPSDO_OLED_SH1106 / _SSD1306 / _SSD1309	OLED 128x64 por I2C (solo uno)
GPSDO_LCD_20x4	HD44780 20x4 vía PCF8574 por I2C
GPSDO_TM1637 / GPSDO_TM1637_6	reloj LED de 4 / 6 dígitos (incompatibles entre sí, y no junto al LCD)
GPSDO_HT16K33	reloj LED de 4 dígitos por I2C (activo por defecto; independiente de la pantalla principal)

Sensores y extras (todos activos de fábrica — comente los que no tenga):

Marca	Hardware
GPSDO_AHT10	AHT10/AHT20 en I2C
GPSDO_BMP280_I2C	BMP280 en I2C
GPSDO_INA219	INA219 en I2C
GPSDO_VCC / GPSDO_VDD	medir el carril de 5 V / 3,3 V
GPSDO_UBX_CONFIG	poner el NEO-6M/7M en modo binario al arrancar
GPSDO_FAKE_UBLOX	clon chino de u-blox: solo sonda de baudrate, cero configuración UBX (véase la Parte 3.2)
GPSDO_GPS_TIMING	soporte de receptores de tiempo (LEA-6T/M8T/NEO-M8T/ZED-F9T): survey-in, qErr
GPSDO_PICDIV	salida de armado del divisor picDIV en PB3
GPSDO_LTIC	el detector de fase por hardware (rampa TIC en PA1) — necesario para los algoritmos 10, 11 y 12
GPSDO_LTIC_ACTIVE_RESET	variante del detector con descarga activa del condensador (apagada para el detector RC clásico de Kaashoek)
GPSDO_PWM_DITHER	tensión de control de 24 bits desde un PWM de 13 bits con dither (no lo apague — este es el buen DAC)
GPSDO_DAC_EXT	DAC SPI externo — un stub que se niega a compilar a propósito
GPSDO_GEN_2kHz_PB5	onda cuadrada de prueba de 2 kHz en PB5
GPSDO_BLUETOOTH	módulo HC-06 en PA2/PA3
GPSDO_BLUETOOTH_PARALLEL	USB y Bluetooth a la vez (activo por defecto)

El archivo se niega a compilar (a propósito) si elige dos pantallas del mismo tipo o dos elementos que se pelean por los mismos pines. Lea el mensaje `#error` — nombra el conflicto.

Deje activos `GPSDO_LTIC` y `GPSDO_PWM_DITHER`, salvo que de verdad no tenga el hardware del detector: sin LTIC pierde los algoritmos 10–12, que son todo el sentido de la v1.06.

1.4 Configurar TFT_eSPI (solo pantallas TFT)

TFT_eSPI se configura en la librería, no en el sketch. Busque `Arduino/libraries/TFT_eSPI/User_Setup.h` y póngale el bloque de su panel:

240x320 (ST7789 o ILI9341):

```
#define ST7789_DRIVER           // or ILI9341_DRIVER
#define TFT_WIDTH  240
#define TFT_HEIGHT 320
#define TFT_MISO PA6           // required on STM32 even if the display has no MISO pin
#define TFT_MOSI PA7
#define TFT_SCLK PA5
#define TFT_CS  PB13
#define TFT_DC   PB12
#define TFT_RST  PB15
#define TFT_RGB_ORDER TFT_BGR
#define TFT_INVERSION_OFF      // fixes inverted colours on some ST7789 modules
#define LOAD_GLCD              // font 1 – frequency readout + splash
#define LOAD_FONT2             // font 2 – header + data grid
#define LOAD_FONT4             // font 4 – status bar, busy messages
#define SPI_FREQUENCY 40000000 // 40 MHz works; drop to 27 MHz for long wires
```

320x480 (ILI9488 / ILI9486):

```
#define ILI9488_DRIVER          // works for both ILI9488 and ILI9486
#define TFT_WIDTH  320
#define TFT_HEIGHT 480
// mismas líneas TFT_MISO/MOSI/SCLK/CS/DC/RST/RGB_ORDER que arriba
#define LOAD_GLCD              // font 1 – splash credits
```

```
#define LOAD_FONT4           // font 4 – splash subtitle path
#define LOAD_GFXFF          // REQUIRED on this panel – free fonts
#define SPI_FREQUENCY 4000000 // don't skimp: this panel pushes 2.4x the pixels
```

Cableado del panel: SCK→PA5, SDI→PA7, RES→PB15, D/C→PB12, CS→PB13. `TFT_MISO PA6` debe estar definido aunque no haya nada conectado a él.

1.5 `build_opt.h` — no lo toque

El archivo contiene tres flags del compilador y no requiere ninguna acción:

```
-DSERIAL_RX_BUFFER_SIZE=256 -DSERIAL_TX_BUFFER_SIZE=512
-DCDC_TRANSMIT_QUEUE_BUFFER_PACKET_NUMBER=16
```

La primera línea agranda los búferes serie para que no se pierdan frases GPS. La segunda amplía la cola de transmisión USB-CDC de 128 bytes a 1 KB, de modo que el informe de telemetría de 1 Hz (unos 400 caracteres) siempre quepa — sin ella, un host que se conecta y no lee podía frenar el informe durante segundos. El núcleo STM32 recoge el archivo automáticamente. No lo borre.

1.6 El menú Tools — placa y librería de C

En Arduino IDE: **Tools** →

- **Board:** "Generic STM32F4 series" → **BlackPill F411CE**.
- **C Runtime Library:** **Newlib Nano + Float Printf/Scanf**. Obligatorio. Sin ello el firmware compila, pero imprime basura donde debería haber un número en coma flotante (todas las tensiones, frecuencias y fases).

1.7 Activar USB CDC (para tener consola por USB)

CDC es lo que convierte `Serial` en un puerto COM virtual por el cable USB. El rótulo de arranque, la línea de comandos y el informe de 1 Hz viajan por ahí. Sin CDC la placa sigue funcionando — la pantalla se actualiza, el lazo disciplina el oscilador — pero por USB parece completamente muerta: sin rótulo, sin prompt, nada.

1. **Tools** → **USB support** (o "USB support (if available)") → "**CDC (generic Serial supersedes U(S)ART)**". El nombre exacto varía algo entre versiones del núcleo; elija la que mencione **CDC** y **generic Serial**.
2. Recompile y regrabe. Cambiar este ajuste cambia el firmware — no es un conmutador en caliente.
3. Tras grabar, **pulse una vez el botón RESET de la placa**. El chip pasa de dispositivo USB DFU (grabación) a dispositivo CDC serie; sin el reset, algunos hosts siguen hablando con el dispositivo antiguo, ya desaparecido.
4. Aparece un puerto COM nuevo (Windows: "STM32 Virtual COM Port", **VID 0483, PID 5740**). Si Windows lo muestra como "Unknown device" o "device descriptor request failed", instale el driver una vez con Zadig — véase también la Parte 2.5.
5. Abra el puerto a 115200. El firmware **espera hasta 3 segundos** tras el reset a que el host abra el puerto antes de imprimir el rótulo — un driver lento no se traga las primeras líneas. Si abre el terminal y no ve nada, pulse RESET otra vez con el terminal ya abierto.

Dos cosas que conviene saber:

- El modo DFU (grabación) y el modo CDC (trabajo) son para Windows **dos dispositivos USB distintos**. Los primeros ciclos de grabación pueden disparar cada uno su ronda de instalación de drivers — es normal, se asienta.
- Con `GPSDO_BLUETOOTH_PARALLEL` compilado (la configuración de fábrica), todo lo que sale por USB se duplica al UART Bluetooth a **57600** — una consola de reserva lista, por si el USB se resiste.

1.8 Compilar y comprobar el tamaño

Sketch → Verify/Compile. Al terminar, lea la línea `Sketch uses NNNNNN bytes`.

NNNNNN debe mantenerse por debajo de 393216. Son 384 KB — todo lo que hay por encima pertenece al anillo de ajustes del sector 7 (Parte 2). Ignore el porcentaje que imprime el IDE: se calcula contra los 512 KB completos

del chip, así que una compilación que «cabe» en porcentaje puede invadir el sector 7. La propia v1.06 ocupa unos 250 KB — margen de sobra, pero no ignore un salto repentino.

Parte 2 — Grabar el firmware, y la regla del sector 7

2.1 Cómo se divide el flash de 512 KB

Sectores	Rango de direcciones	Contenido
0 – 5	0x08000000 – 0x0803FFFF	Su firmware (384 KB)
6	0x08040000 – 0x0805FFFF	reserva (margen de crecimiento)
7	0x08060000 – 0x0807FFFF	el flash ring: ajustes + calibración + datos aprendidos

En el sector 7 vive todo lo que el aparato ha aprendido de sí mismo:

- sus ajustes guardados (algoritmo elegido, todos los parámetros del lazo, zona horaria),
- la calibración de sensibilidad del oscilador hecha por **CT**,
- la calibración del detector de fase hecha por **LC**,
- el modelo aprendido de deriva/amortiguación y el último punto de trabajo.

Reconstruir todo eso cuesta una hora de banco. Proteger el sector 7 es por tanto **la regla operativa más importante de todo el proyecto**.

2.2 La regla de oro

Una grabación normal nunca toca el sector 7. El borrado completo del chip lo destruye.

- La subida desde Arduino IDE, DFU y el bootloader STM32duino reprograman solo los sectores 0–5. Sus ajustes sobreviven cada actualización rutinaria.
- «Erase Chip» en ST-LINK Utility / STM32CubeProgrammer, o erase sin rango de direcciones en J-Link, borra el chip entero **incluido el sector 7**. Nunca lo use en una unidad funcionando y calibrada.

Si graba a mano con ST-Link o J-Link, borre **solo** el rango del firmware y cargue después:

```
> erase 0x08000000 0x0803FFFF
> loadbin firmware.bin 0x08000000
```

Si el sector 7 llega a perderse: nada se rompe. En el siguiente arranque el firmware detecta el anillo en blanco, lo formatea y arranca con valores por omisión — simplemente repite **CT**, **LC**, ajusta su algoritmo y zona horaria, y **ES**. Pierde el ajuste fino, no el aparato.

Antes de la primera grabación de una unidad terminada, haga una copia completa (ejemplo con J-Link; ST-LINK Utility tiene el botón «save» equivalente):

```
> JLinkExe -device STM32F411CE -if SWD -speed 4000
> savebin backup_full.bin 0x08000000 0x800000
> exit
```

Restaurar: `loadbin backup_full.bin 0x08000000`.

2.3 Grabación, método A — ST-Link (el más simple)

1. Conecte el ST-Link V2 a los pines SWD del BlackPill (SWDIO, SWCLK, GND, 3V3).
2. Arduino IDE → Tools → **Upload method: "STM32CubeProgrammer (SWD)"**.
3. Pulse Upload. Listo. El programador escribe solo lo que ocupa el sketch.

2.4 Grabación, método B — DFU por USB (sin programador)

¿Qué BOOT0 tiene su placa? Los BlackPill de WeAct antiguos llevan un jumper BOOT0; los actuales (v3.1) solo un botón BOOT0. Use la receta que corresponda.

Placas con jumper: ponga el jumper BOOT0 a 1 y pulse RESET — la placa aparece como "STM32 BOOTLOADER". Tras la subida, devuelva el jumper a 0 y pulse RESET.

Placas con botón (v3.1), la forma fiable: desconecte cualquier alimentación externa; **pulse y mantenga BOOT0**, conecte el cable USB **manteniendo el botón**; a los pocos segundos suelte BOOT0 — Windows no debería avisar de «dispositivo no reconocido» y STM32CubeProgrammer verá la placa. Suba el firmware y pulse RESET. (Mantener BOOT0 mientras se pulsa NRST, como sugieren otras páginas, normalmente **no** funciona — sobre todo con la placa ya montada en un PCB.)

Windows puede pedir un driver la primera vez — véase la nota siguiente.

2.5 Tras cualquier grabación: la nota del USB

- Cuando el IDE termine, **pulse el botón RESET de la placa** — el dispositivo USB re-enumeración limpiamente del modo bootloader al CDC serie del firmware.
- El serie USB del firmware es **VID 0483, PID 5740** ("STM32 Virtual COM Port"). Si Windows se niega a abrirlo, instale el driver una vez con Zadig.

Parte 3 — Primer arranque: calibración, algoritmo, guardado

Hace falta un programa de terminal (PuTTY, Tera Term, el monitor serie del Arduino IDE o el sintonizador de la Parte 8 — el sintonizador es el más cómodo).

3.1 Conexión

1. Conecte el USB (y/o el Bluetooth a 57600 — en la configuración de fábrica ambos funcionan a la vez).
2. Abra el puerto a **115200, 8N1**. Fines de línea: CR o LF, ambos valen.
3. Pulse **Intro**. Debería ver el prompt o el inicio del informe de 1 Hz.
4. Escriba **H** y pulse Intro. Se desplaza la lista completa de comandos — es la Parte 7 de este manual, en vivo.

El registro de arranque, línea a línea. Los primeros diez segundos tras el reset imprimen una secuencia fija. Este es uno típico de una unidad sana ya configurada (las líneas exactas dependen del hardware), y después, qué significa cada una:

```
=====
GPSDO v1.06-rtos compiled 2026-08-23 14:32:45
FreeRTOS port by J. M. Niewinski with Claude, GLM-5.3 Max & Qwen3.8-Max AI
https://github.com/jmnlabs/GPSDO_FreeRTOS
Inspired by GPSDO v0.06c by Andre Balsa
https://github.com/AndrewBCN/STM32-GPSDO
Algos 0-2 original design by Andre Balsa
Algos 3-9 by J. M. Niewinski
Algo 10 (LTIC 3-stage) inspired by Dan Wiering's measurements
Algo 11 (LTIC-Lars) after Lars Walenius' PI loop
Algo 12 (multi-level accumulator) after Alan Cashin (MIS42N)
Type H = help SW = stack diagnostics
=====
Reset cause: POWER-ON/BROWN-OUT PIN/NRST
Flash ring: sector 7 ready
Settings: recalled from flash ring
Live store: LRN + LC applied from flash ring
Initial PWM=44653 algo=12 time_offset_min=120
GPS init: probing baud rate...
GPS detected at 38400
GPS: disabling noisy NMEA at 38400 baud
GPS: 4/4 NMEA sentences disabled
GPS: sending UBX config at 38400 baud
UBX: CFG-NAV5 ACK
LEA-T: accepted CFG-TMODE2 (28B)
Hardware configured, creating RTOS objects...
Timers started
Starting FreeRTOS scheduler
HW: AHT10/AHT20 sensor OK (I2C 0x38)
HW: BMP280 sensor OK (I2C 0x77)
HW: INA219 sensor OK (I2C 0x40)
HW: LTIC phase input OK (PA1 analog)
TFT: init start (SPI1 PA5/PA7, CS=PB13 DC=PB12 RST=PB15)
TFT: freq-band sprite (4-bit) created
TFT: header sprite (4-bit) created
TFT: data sprite (1-bit) created
```

Después arranca el informe de 1 Hz (Parte 6). Lo que dice cada línea:

Línea	Significado
rótulo (GPSDO v1.06-rtos ...)	identidad del firmware. Si ve caracteres basura: velocidad equivocada — use 115200
Reset cause:	por qué se reinició el chip. POWER-ON/BROWN-OUT PIN/NRST = encendido o botón normales. SOFTWARE = reset ordenado por el firmware (RB, fin de una subida). INDEP-WDG/WINDOW-WDG = watchdog (este firmware no usa ninguno — trátelo como señal de fallo). La línea extra -> supply dipped: check the 3V3 rail under load aparece tras un brown-out: el carril de 3V3 cayó bajo la carga del OCXO — arregle la alimentación, no lo ignore

Línea	Significado
Flash ring: sector 7 ready	el anillo de ajustes del sector 7 se encontró válido
Flash ring: sector 7 blank/formatted (defaults)	anillo virgen o borrado — normal en la primera grabación ; el firmware lo formatea y usa los valores de compilación
Settings: recalled from flash ring	sus ajustes guardados (algoritmo, parámetros del lazo, zona horaria, flags) se aplicaron
Settings: none stored (compile-time defaults)	nada guardado aún — lo esperable en una unidad nueva; recorra la Parte 3 y ejecute ES
Live store: LRN + LC applied from flash ring	datos aprendidos (calibración LC, modelo de deriva, último punto de trabajo) aplicados — la línea solo aparece con una unidad ya calibrada
Initial PWM=... algo=... time_offset_min=...	punto de trabajo elegido al arrancar: código PWM final (los datos aprendidos pisan los ajustes si son más recientes), algoritmo restaurado y desfase de zona horaria en minutos
GPS init: probing baud rate... / GPS detected at ...	el receptor se encuentra y se mide su velocidad (prefiere 38400)
GPS: no response to baud probe, defaulting to 9600	ningún receptor respondió — revise el cableado del módulo GPS antes que nada
GPS: disabling noisy NMEA... / GPS: sending UBX config... / UBX: CFG-NAV5 ACK	el receptor pasa a modo binario y dinámica estacionaria; las variantes NAK/no response son supervivibles — el receptor simplemente conserva su configuración
LEA-T: accepted CFG-TMODE2	un receptor de tiempo (clase LEA-M8T) quedó configurado en survey-in / Time Mode — solo aparece con GPSDO_GPS_TIMING y receptor de tiempo
Hardware configured... / Timers started / Starting FreeRTOS scheduler	arranque interno; las tres deben aparecer siempre
HW: <sensor> OK (I2C ...) / HW: <sensor> not found	el escaneo del bus I2C: cada sensor informa presente o ausente. Un sensor ausente no es fatal — la unidad funciona sin él, solo ese campo queda vacío en los informes
HW: LTIC phase input OK (PA1 analog)	la entrada del detector de fase quedó configurada (los algoritmos 10–12 dependen de ella)
TFT: init start (...)	comienza el arranque de la pantalla. Si tras esta línea no sigue nada , el cableado TFT o el User_Setup.h está mal (Parte 1.4) — es el caso «pantalla blanca/muerta pero el serie vive»
TFT: ... sprite FAILED – direct-draw fallback	la pantalla funciona pero redibuja más despacio (faltó RAM para los sprites antiparpadeo) — cosmético, no fatal

Líneas que aparecen **más tarde**, con el aparato funcionando, y su significado:

Línea	Significado
LTIC: running UNCALIBRATED (run LC)	hay un algoritmo 10/11/12 activo pero LC nunca se ejecutó — los números de fase son nominales, no medidos. Ejecute LC

Línea	Significado
LTIC ACQ: polarity unset – run 'LPOL -1' (or +1)	el algoritmo 10 se niega a actuar hasta que fije la polaridad (espera con seguridad en vez de adivinar). Los algoritmos 11 y 12 asumían +1 y actuaban igualmente; ahora esperan igual
picDIV: armed (output stopped, waiting for 1PPS sync)	el divisor quedó armado; un hueco de 1–1,2 s en su salida es lo esperado antes de sincronizar al siguiente flanco de PPS
LEA-T: ... survey ... % / s) – continuing anyway	monitor del survey-in; «continuing anyway» significa que el survey agotó el tiempo por encima del objetivo de precisión — normalmente cielo tapado
!!! FreeRTOS ... (configASSERT / STACK OVERFLOW / MALLOC FAILED)	el firmware capturó un fallo, con archivo/línea o nombre de tarea — el LED azul parpadea rápido al mismo tiempo. Anote ambas cosas; la unidad necesita reset (RB)

Una unidad virgen pide calibración automáticamente en el primer arranque — puede ver la cuenta atrás de calibración antes que cualquier otra cosa.

3.2 Deje que el GPS se asiente

- Dé a la antena una vista limpia del cielo. Un alféizar sirve; una antena geodésica en la azotea sirve mejor.
- Con un receptor de tiempo (M8T/F9T...) y `GPSDO_GPS_TIMING` compilado, al arrancar corre el **survey-in**: el receptor mide su propia posición durante al menos 300 s hasta que la estimación baja de 5 m, y entonces pasa a **Time Mode** en posición fija. Desde entonces la pantalla muestra `HDOP:TIME` en vez de un número. El survey-in solo debe completarse una vez; se repite tras perder alimentación. (`SV 0` lo apaga, `SV 1` lo reactiva, en el siguiente arranque.)
- El calentamiento del OCXO tarda 300 s (`WU 0` lo salta; déjelo activo).
- El LED amarillo está **apagado sin fix GPS, fijo con fix** — cuando algo se vea mal, mire ahí primero.

¿Qué módulo GNSS elegir? Un receptor de tiempo u-blox genuino (clase LEA-M8T) es la base para la que está construido este firmware: survey-in, Time Mode, la corrección de diente de sierra `qErr`. También disciplinará con módulos de navegación baratos —incluidos los clones chinos de u-blox de eBay/AliExpress— porque al lazo le basta el pulso 1PPS y las frases NMEA. Pero los clones ignoran la configuración binaria: espere `0/4 NMEA sentences disabled`, tramas CFG sin ACK y unos ~15 s más de arranque mientras expiran los timeouts; no hay survey-in ni `qErr`, la pantalla muestra un HDOP numérico para siempre, y el vagabundeo extra de la posición se cuelga en la fase — justo lo que absorben los límites holgados por omisión del algo-12. Una rareza conocida de los clones: tras la ronda de configuración fallida, algunos dejan de emitir datos y la pantalla se queda en "acquiring" — la cura es entrar en el túnel de u-center (`T` con el baud del módulo, p. ej. `T 9600`) y simplemente dejar que expire: la nueva sonda tras el túnel restaura el funcionamiento. Un conmutador de firmware para Un conmutador de firmware lo cubre todo: defina `GPSDO_FAKE_UBLOX` en `gpsdo_config.h` y el firmware solo sondea el baudrate y no envía nada más — un solo interruptor, al margen de las demás opciones de GPS activas. Aún sin probar en hardware (el autor no tiene ningún módulo clon); se agradecen informes de campo.

3.3 Calibración — primero CT, después LC. El orden importa.

Estos dos comandos enseñan al firmware dos hechos físicos distintos, y el segundo necesita al primero:

Paso 1 — CT (sensibilidad del oscilador + ajuste del lazo, ~3 minutos). Escriba `CT` e Intro. El firmware lleva la tensión de control por tres puntos (1,5 V; 2,0 V; 2,5 V), mide la frecuencia en cada uno, ajusta una recta y calcula **K** — **cuántos hercios vale un paso PWM en su oscilador** (rango aceptado 0,02–2 mHz/LSB). Con K deriva ganancias PID razonables para los algoritmos 3–9 y los lazos LTIC, y **guarda él solo el grupo PID** al flash. No corte la alimentación durante esos tres minutos.

Paso 2 — LC (calibración del detector de fase, ~5 minutos). Escriba `LC` e Intro. El firmware arma el divisor picDIV, centra el detector, recorre una rampa del condensador y mide los **ns por voltio**, el **offset de cero en voltios** y el **rango en ns** del detector. Con `PASS` se **guarda solo**. Se niega a ser útil si `CT` no ha corrido — por eso el orden importa.

Comprobación: escriba `LL` y confirme que el bloque LTIC muestra `ns_per_volt`, `zero_offset`, `range_ns` distintos de cero. Escriba `DAC` — hasta le dirá cuántos microhercios vale un paso de salida.

3.4 Elegir el algoritmo

- **LA 12** — el acumulador multinivel (según Alan Cashin). El buque insignia de la v1.06: sujeta la fase a unos pocos ns RMS, corrige de promedio cada pocos minutos. **Es la recomendación.**
- **LA 11** — el lazo PI continuo de Lars Walenius. El clásico maduro y suave; excelente comportamiento a largo plazo con una sola perilla (**LTC**).
- **LA 10** — lazo de fase en tres etapas (ACQ→DPLL→LOCK). Sólido, con más parámetros.
- Los algoritmos 0–9 son la colección histórica — funcionan, están congelados, véase la Parte 4.

LA a secas muestra el algoritmo actual. Elegir **no guarda** — véase el paso 3.6.

3.5 Extras recomendados

- **SAW 1** — corrección de diente de sierra. El error de cuantización del 1PPS del receptor ($\pm 8... \pm 25$ ns, reportado cada segundo como **qErr**) se resta de la fase medida. Con un receptor de tiempo es precisión gratis; actívelo.
- **TZ <ciudad>** — hora local con DST (p. ej. **TZ Madrid**, **TZ Adelaide**), o **T0 1** para desfase fijo, **LT 1** para mostrar hora local en vez de UTC. **H TZ** explica el formato de regla si falta su zona.
- **P0 <Pa>** — offset añadido a la lectura de presión del BMP280 (–5000..5000).
- **A0 <m>** — offset añadido a la **altitud del GPS** en pantalla y telemetría (–3000..3000 m). Corrige la altitud mostrada a la realidad del terreno, p. ej. si el modelo de geoide se equivoca 30 m: **A0 30**.

3.6 Guardar todo — ES

Escriba **ES** e Intro. Esto escribe **todos** los ajustes más los datos aprendidos al flash ring del sector 7. Desde este momento un corte de corriente no pierde nada.

Un puñado de comandos-preferencia se guardan **ellos solos** en el momento de fijarlos y se lo dicen — la respuesta trae **[auto-saved: ...]**. Son: **TZ**, **T0**, **LT** (grupo zona horaria), **WU**, **SPL**, **SV** (flags), **P0**, **A0** (offsets), además de **CT** (guarda él solo el grupo PID que acaba de derivar) y un **LC** aprobado (se guarda al anillo live).

Todo lo relativo al **lazo** — ganancias (**KP/KI/KD/IL**), todos los parámetros LTIC y Lars, los ajustes del algo-12, y la propia elección **LA** — vive en RAM hasta que lo guarde. Cada uno de esos comandos responde con una pista tipo **[not saved – run 'ES LTIC' to keep it]**: ejecute el guardado de grupo indicado, o simplemente recuerde: **tras una sesión de ajuste, ES**.

3.7 Cómo se ve un aparato sano

- La palabra de tendencia (pantalla + línea **PWM**: del informe) llega a **LOCK** y se queda — con el algo 12, destellos breves de **CORR/ZC** son **normales y sanos** (una corrección es el algoritmo auto-comprobándose con calendario; la pantalla aun así los cuenta como enganchado).
- La fase (**dph**: del informe) deambula en torno a cero dentro de decenas de ns y siempre vuelve — no puede irse de rampa.
- Las ventanas de frecuencia se cierran: **100s**: en milihercios, **1ks**: en microhercios, en una o dos horas.
- **CS** (estadística de correcciones) imprime números RMS pequeños y estables que no crecen hora a hora.
- En el TFT, el número grande de frecuencia está **verde**.

En un banco tranquilo con el algo 12, espere: fase 5–20 ns RMS, correcciones de unos pocos LSB cada pocos minutos, desviación de Allan en la clase 1e-11...1e-12 desde 1000 s hacia arriba. Junto a un radiador, una puerta que se abre o el sol directo los números serán peores — eso es física, no un fallo.

Parte 4 — Los trece algoritmos (0–12) y sus parámetros

Una idea subyace a todo esto: el firmware cuenta la frecuencia del OCXO con un contador enventanado (TIM2), mide la fase GPS-contra-OCXO con el detector LTIC y ajusta la tensión de control (el «DAC» PWM, 65536 pasos, ~48,8 µV por paso a 16 bits; con `GPSDO_PWM_DITHER` la resolución efectiva son 24 bits — unos 0,2 µV por paso). Convención de signo: error medido `e = freq - 10 MHz`; si `e > 0` el oscilador va rápido y el PWM debe **bajar** (para un EFC de sensibilidad positiva; `LPOL` / la gestión de polaridad cubre los EFC invertidos).

Si fase, error de frecuencia o PID son palabras nuevas para usted, deténgase aquí y lea el Apéndice A (qué persigue el lazo y por qué debe ser suave) y el Apéndice B (qué hacen de verdad las letras P, I y D). Diez minutos ahí vuelven legible cada parámetro de abajo.

La línea `Learn:` del informe nombra el algoritmo activo; el sintonizador elige de ahí la familia de gráficas automáticamente.

4.0 El menú

#	Nombre (como se muestra)	En una frase	Estado
0	primitive	el controlador escalonado original de André Balsa, ciclo 429 s	por defecto tras un flash frío
1	forced-drift	+1 LSB por 1000 s — caracterización del oscilador	diagnóstico
2	random-walk	ruido ±1 LSB cada 5 s — medición del suelo de ruido	diagnóstico
3	FLL-PID-man	PID sobre la media de 100 s	clásico
4	PLL-PI-man	PI sobre fase, ciclo 10 s	clásico
5	PLL-PID-man	como el 4 con su propio hueco de ajuste	clásico
6	FLL-PID-gen	FLL PID, ajuste por algoritmo genético	clásico
7	PLL-PID-gen	el viejo PLL de batalla; su Kp almacena el resultado de CT	clásico
8	hybrid-FLL-PLL	mezcla sigmoide del 6 y el 7 según el tamaño del error	clásico
9	NN-MLP	red neuronal pequeña + conducción térmica aprendida en holdover	clásico
10	LTIC-3stage	máquina de estados ACQ→DPLL→LOCK sobre el detector de fase	línea recomendada
11	LTIC-Lars	lazo PI continuo de Lars Walenius	línea recomendada
12	multi-level	acumulador multinivel de Alan Cashin	buque insignia

Los algoritmos 0–9 están **congelados**: permanecen porque funcionan y porque el hueco de ajuste del 7 almacena además la calibración CT. El desarrollo nuevo es solo 10/11/12.

Selección: `LA <n>` (persistente con `ES ALGO`). Una unidad recién flasheada arranca siempre con el algoritmo 0 — elija el suyo una vez, guarde, y queda para siempre.

4.1 Algoritmos 0–2 (diagnóstico)

Sin parámetros. El 0 mueve el PWM cuando las medias de frecuencia cruzan umbrales fijos. El 1 lleva el PWM en rampa lineal (caracterizar el EFC). El 2 inyecta ruido (medir el suelo del lazo). No los necesitará en uso normal.

4.2 Algoritmos 3–9 (los clásicos) — parámetros por KP/KI/KD/IL

`KP <algo> <val>` / `KI` / `KD` fijan las ganancias (algoritmos 3–7, 0..100000); `IL <algo> <val>` la pinza del integrador (algoritmos 3–9, 100..100000). `LP [n]` lista. Valores por omisión (y la base de la que parte `CT` para derivar los suyos medidos):

algo	Kp	Ki	Kd	I_LIMIT
3	70,0	0,70	175,0	9000
4	1000	0,020	2,0	7000
5	1000	0,020	2,0	10000
6	205	0,264	14950	13000
7	1000	0,020	2,0	10000
8	—	—	—	13000
9	—	—	—	450

Extras: el algoritmo 8 tiene **BC** (cruce de mezcla, por defecto 0,024 Hz) y **BS** (escala de mezcla, 0,012 Hz) — el centro y el ancho de la sigmoide que decide cuánto FLL contra PLL mezclar a cada error; el algoritmo 9 tiene **NS** (paso máximo, por defecto 175 LSB). Guardar: **ES PID**. Sinceramente: tras **CT** no debería necesitar tocar nada — para eso está CT.

4.3 Algoritmo 10 — LTIC de tres etapas

Máquina de estados sobre el detector de fase: **ACQ** (captura guiada por frecuencia, además recentra la rampa del detector) → **DPLL** (ciclo 2 s, frecuencia + fase) → **LOCK** (correcciones cada **LIV** segundos con la estimación de par de ventanas).

Parámetros (todos **ES LTIC**; **LL** lo lista todo):

Comando	Significado	Rango / omisión
LAT	umbral de fase de ACQ	ns, por defecto 100
LDT	umbral de deriva DPLL→LOCK	1e-13..1,0, por defecto 5e-10
LIV	intervalo de corrección en LOCK	1..600 s, por defecto 300
AQP/AQI/AQD/AQL	PID de la etapa ACQ	0..100000
DPP/DPI/DPD/DPL	PID de la etapa DPLL	0..100000
LKP/LKI/LKD/LKL	PID de la etapa LOCK	0..100000
LPOL	polaridad PWM→fase, -1/0/+1	0 = auto
LCV	objetivo de centrado de ACQ	0 (= auto, centro del rango) .. 3,3
ACG g [cap]	ganancia [tope de paso] del centrado	50..20000 LSB/V [5..1000 LSB]
FA/FAD/FAL	ventana de media de frecuencia del término amortiguador (ambas / DPLL / LOCK)	10, 100 o 1000 s

ltic_autotune (parte de CT/LC) rellena las ganancias por etapa a partir de la sensibilidad medida, así que la secuencia de arranque descrita ya ajusta este algoritmo. Si ACQ nunca sale, revise **LPOL** — el lazo avisa y espera hasta que lo fije.

4.4 Algoritmo 11 — LTIC-Lars, lazo PI continuo

El lazo de Lars Walenius, portado con fidelidad. Un controlador PI evaluado cada segundo, con prefiltro adaptativo; lock cuando la fase filtrada se mantiene dentro de **LPL** ns durante **LTC** × **LPF** segundos. Detector al riel → captura automática por frecuencia, con un rearmado de picDIV permitido.

Comando	Significado	Rango / omisión
LG	ganancia del lazo. 0 = auto desde CT (recomendado)	0..10000
LD	amortiguación	>0..1000, por defecto 3
LTC	constante de tiempo del lazo — la única perilla	1..600 s, por defecto 60
LFD	divisor del prefiltro (prefiltro = LTC/esto)	1..100, por defecto 2

Comando	Significado	Rango / omisión
LTO	objetivo de fase, cuentas ADC del detector	0..4095, por defecto 2620
LPL	ventana de fase del lock	1..10000 ns, por defecto 100
LPF	factor de retención del lock (hold = LPF × LTC)	1..100, por defecto 5
LTK	coeficiente térmico, pasos DAC por paso ADC	-32000..32000, 0 = apagado
LTR	referencia de temperatura, cuentas ADC	0..4095

El ajuste en una frase: deje **LG 0**, ponga **LTC** según lo suave que quiera el lazo (60 s es buen arranque; 240 s apacigua un sitio ruidoso; más corto solo añade ruido), y el resto por omisión. Todo con **ES LTIC**. La pestaña **LTIC-Lars** del sintonizador expone exactamente estos campos.

4.5 Algoritmo 12 — el acumulador multinivel (buque insignia)

Puerto del GPSDO de Alan Cashin (MIS42N), refinado durante v1.04–v1.06. Sin ciclo fijo: **el tamaño del error elige el tiempo de promediación**.

Cómo funciona, en palabras llanas. Cada segundo la lectura de fase entra en una escalera de acumuladores. El nivel n promedia $2^{(n+1)}$ segundos — la escalera corre 2, 4, 8, ... 2048 s (11 niveles). Cada nivel guarda un par de valores (A = mitad vieja, B = mitad nueva). Al cerrarse el par se calculan dos números: la **pendiente** ($B - A$ = error de frecuencia en ese tramo) y la **fase extrapolada** ($3B - A$ = la fase ahora mismo). Si la fase cabe en el límite del nivel, el par se suma y asciende un nivel (doble promediación). Si lo excede — corrección inmediata, la escalera se reinicia, y la corrección se reparte suavemente en el tramo del que vino (mínimo 64 s — «un GPSDO quiere que un error grande se corrija con suavidad»; el suelo original de Alan es 16 s, elegido para que una corrección en caliente no exija una tensión de control fuera del rango del DAC — un suelo menor da pasos mayores, uno mayor da una recuperación de fase más lenta). Independientemente de eso, alcanzar el nivel **MR** (por defecto 9 = 1024 s) fuerza una corrección aunque todo esté bajo los límites — si no, una deriva lenta jamás recibiría respuesta.

Cada corrección lleva hasta tres partes, y es a propósito: cancelar el error de **frecuencia** medido, devolver la **fase** a cero, y nunca una sin la otra. Después, el truco que Alan llama esencial: el empuje deliberado queda apuntado, y a la primera lectura de fase que cruza cero en la dirección esperada se retira exactamente ese empuje (**ZC**, cancelación en el cruce por cero) — el oscilador queda con la frecuencia correcta y sin error de fase.

Tendencias propias del 12: **WAIT** (aún sin datos) → **SYNC** (5 s de asentamiento tras armar el divisor) → **FLL** (detector al riel, captura por frecuencia) → **NOPH** (fase no válida, PWM congelado) → **ACQ** → **LOCK** (>16 s de quietud genuina y frecuencia en puerta; las correcciones y los **ZC** no reinician el contador de quietud — son salud, no ruido). Los destellos transitorios **CORR** / **ZC** son normales. **NoCT** significa: ganancia en auto pero **CT** nunca corrió — ejecute **CT**. **NoPL** significa: polaridad del EFC sin fijar — el lazo espera hasta que ponga **LPOL -1** o **1**.

Parámetros (**ES ALG012**; **ML** lista todo, incluida la estimación de ruido 1-sigma en vivo):

Comando	Significado	Rango / omisión
MG	ganancia, LSB por ns de fase. 0 = auto desde CT	0..10000, por defecto 0 (auto)
MR	nivel que fuerza una corrección	0..10, por defecto 9 (1024 s)
MF	origen de los límites por nivel: 0=seguir MG, 1=tabla guardada, 2=fórmula sigma, 3=ajuste medido	0..3, por defecto 0
MFT	con MF 3: segundos objetivo entre correcciones por ruido	0 (=3600) o 2048..65535 s
MLP n v	una fila de la tabla de límites de 11 niveles	nivel 0..10, valor 1..500000 u. de acumulador

La tabla de límites por omisión es la del propio Alan (reescalada de su detector de 25 ns a este de ~1 ns). En nanosegundos por tramo: L0 462, L1 400, L2 331, L3 264, L4 191, L5 164, L6 126, L7 117, L8 108, L9 103, L10 63 ns. **ML** la marca **UNTUNED** (solo la fila de 128 s derivó alguna vez de una especificación) — trátela como el punto de partida

probado, no como escritura. Su origen, en palabras del propio Alan, es un **escenario de recepción en el peor caso**: los números se calcularon para un NEO-6 con antena dentro de una habitación, cuyo 1PPS puede desviarse ± 100 ns en ratos cortos (± 30 – 40 ns de promedio). La tabla es holgada a propósito — dimensionada para el peor receptor posible, no para el hardware clase LEA-M8T que esta construcción suele llevar.

¿Qué MF elegir? Resultados de campo, dos placas:

- **Sitio tranquilo (antena de alféizar en casa, temperatura estable)**: la fórmula sigma (MF 2) o la medida (MF 3) funcionan — el lazo se autoescala y corrige cada pocos minutos.
- **Sitio ruidoso (taller con puerta, cerca de un radiador, aire en movimiento)**: el autoescalado puede cazar en el nivel 0 (límites más estrechos que las deambulaciones reales del entorno). Allí, la tabla holgada de Alan (MF 1) fue dramáticamente más estable. Si ve correcciones de nivel 0 constantes, cambie a MF 1.

Detrás de ambos resultados está el principio de diseño declarado por Alan, conveniente conocer antes de ajustar: «*no queremos que el algoritmo trabaje con límites, queremos que trabaje con correcciones programadas a intervalos definidos*». La corrección por nivel de ejecución (MR) debe ser el motor, y los límites solo una red de seguridad para el calentamiento y las excursiones de recepción — y exactamente por eso su tabla es holgada. Una tabla estrecha que dispara sin parar trabaja, según esa filosofía, contra el algoritmo y no con él.

La ganancia (MG) pertenece al **oscilador**; los límites pertenecen al **ruido del sitio** — por eso los fijan comandos distintos, y para eso está MF: para poder expresar «ganancia medida, límites a mano».

En el sintonizador: la pestaña **Multi-level (algo 12)** tiene los cuatro escalares y la tabla completa de límites de 11 filas con botones Send/Read/List, y las gráficas en vivo cambian solas a la familia del algo-12 (ph error de fase, nivel, contador de correcciones, sigma, contador de ZC).

Parte 5 — Pantallas: qué significa cada campo

Pueden funcionar varias pantallas a la vez (I2C + SPI no chocan). Todas muestran la misma verdad con distinto nivel de detalle.

5.1 La TFT (480x320 o 320x240)

```
y= 0..23   barra de título: "GPSDO v1.06"           LMT 14:32:45 Thu
y= 30..62   FREQUENCY — dígitos grandes, con color
y= 70..151  rejilla de datos, dos columnas
y=156..195  fila de sensores
y=204..239  barra de estado
```

El color del número grande de frecuencia es el control de salud de un vistazo:

Color	Significado
verde	enganchado (para 10/11: tendencia LOCK; para 12 también durante CORR/ZC — un lazo que corrige es un lazo sano)
blanco	ajustando
naranja	holdover
rojo	sin señal / sin fix

Durante los procedimientos de arranque, el número se sustituye por cuentas atrás naranjas: Survey <s> ±<m>, OCX0 warmup <s>, Tune <s> (CT), LTIC cal <s> (LC), Calibrate <s>.

Rejilla de datos: columna izquierda — hora UTC + día de la semana, fecha, uptime (contado desde el 1PPS del GPS, no deriva), Algo: n <tendencia>, código PWM: + tensión Vct:. Columna derecha — Sat: + HDOP: (o HDOP: TIME tras el survey-in — esa palabra es *buena*, significa modo de tiempo en posición fija), Lat:/Lon: (6 decimales), Alt: + qErr: (el valor de diente de sierra que se resta ahora mismo), INA: tensión y corriente de alimentación. Filas de sensores: BMP: temperatura + presión, AHT: temperatura + humedad, Vph:/dph: tensión del detector y fase en ns (ovf = rampa fuera de su banda válida), Vcc:/Vdd: carriles de alimentación.

Barra de estado (fondo de color): verde DISCIPLINED FIX OK, naranja HOLDOVER (manual), rojo HOLDOVER (fix lost) o WAITING FOR GPS FIX; con SURVEY añadido mientras corre el survey-in.

5.2 La OLED (128x64) — dos páginas que alternan cada 10 s

Página A (GPS): hora local, frecuencia, Lat/Lon/Alt+Sats, uptime, UTC+temperatura AHT, PWM + tendencia (una H/A parpadeante al borde derecho = holdover manual/auto). Página B (sensores): filas BMP/AHT/INA, Sat+HDOP, UTC. Durante los procedimientos, la fila de frecuencia muestra F SVIN <s>s <m>m, F WARMUP <s>s, F CAL <s>s.

5.3 El LCD 20x4

Línea 0 frecuencia; línea 1 UTC + días de uptime; línea 2 rota cada 10 s (coordenadas / sats+HDOP / AHT / INA / BMP); línea 3 PWM + Vctl + tendencia con el marcador parpadeante de holdover.

5.4 Los dígitos de reloj (TM1637 / HT16K33)

Hora local HH:MM (los dos puntos parpadean al segundo). Todo rayas = arranque/sin datos; oooo = sin fix; molinillos = calentamiento / survey / calibración.

5.5 Los LED de la placa

LED	Significado
Amarillo (PB8)	APAGADO = sin fix GPS · fijo = fix OK · parpadeo lento (1 s) = holdover manual · parpadeo rápido (200 ms) = fix perdido, holdover automático

LED	Significado
Azul (PC13)	solo fallos — parpadeo rápido significa que el firmware capturó una avería (aserción / desbordamiento de pila / memoria agotada), con el motivo por el serie. <i>No</i> es un latido; un LED azul apagado es el estado bueno.

Parte 6 — Telemetría serie (el informe de 6 líneas)

Una vez por segundo (una por PPS) el firmware imprime un bloque de estado con esta forma (modo legible, RH):

```
Up: 000d 02:15:33 UTC: 22/8/2026 14:32:45
Lat: 51.477928 Lon: -0.001531 Alt: 46.5m Sat:10 HDOP:TIME
Freq: 1000000.0000 Hz 10s:0.0 100s:0.02 1ks:0.000 10ks:0.0000
PWM:44653 Vctl:1.970V hit
Learn: algo=11 (LTIC-Lars) gain=auto scale=46 phase=12.3ns LOCK qErr=-8.2ns
BMP:23.4C 1013.2hPa AHT:22.1C 45.3%RH INA:12.05V 250mA Vphase:3.077V dph:1390.5ns
```

(La posición del ejemplo es el Real Observatorio de Greenwich — longitud cero por definición, un sitio público a propósito. Su unidad imprimirá por supuesto sus propias coordenadas; si comparte registros, recuerde la opción **Redact position** del sintonizador.)

Línea a línea:

Línea	Campos
1	uptime (contado desde el PPS, fiable desde la v1.06) + fecha/hora UTC del GPS
2	posición (6 decimales), altitud (altitud GPS más su A0), satélites, HDOP — o la palabra TIME cuando el receptor de tiempo está en modo de posición fija; sin fix → GPS: no position fix yet
3	frecuencia contada en bruto (o --- antes del primer conteo) y las ventanas promediadas de error 10 s / 100 s / 1 ks / 10 ks — cada una aparece solo cuando su búfer se ha llenado
4	código PWM de 16 bits, tensión de control medida, palabra de tendencia (hit, ACQ, DPLL, LOCK, PLL, CORR, ZC, NOPH, NoCT, ...) — en holdover, [HOLDOVER] sustituye a la tendencia
5	línea Learn según el algoritmo (abajo) + qErr cuando SAW 1 está activo
6	temperatura/presión BMP280 (bruta + su P0), temperatura/humedad AHT, tensión/corriente INA, tensión del detector Vphase, fase dph en ns (con el diente de sierra restado, igual que el propio lazo)

La línea Learn por familia:

- algo 11: gain=auto|<val> scale=<n> phase=<ns> LOCK|acq
- algo 12: ph=<ns> level=<n> corr=<n> arm=<n> sig=<ns> zc=<n> secs=<n> — fase acumulada, último nivel que actuó, correcciones, armados, estimación de ruido en vivo, cruces por cero, segundos
- algo 10: state=ACQ|DPLL|LOCK
- algos 3–9: deriva/pendiente/amortiguación aprendidas; el 9 añade el coeficiente térmico aprendido

Modo con tabuladores (RD) imprime una línea por segundo con los mismos datos como columnas (uptime, ventanas de frecuencia, sats, HDOP, PWM, tensiones, todos los sensores, el TIC en bruto) — el formato para volcar a una hoja de cálculo. RP/RR pausa/reanuda. El informe no bloquea: un host que conecta y no leerá pierde colas de informes, pero no congela las pantallas (arreglado en v1.06 — una cola CDC llena llegó a clavar de por vida la tarea de pantalla).

Parte 7 — Referencia de comandos (todos)

Serie, 115200, sin distinguir mayúsculas, terminado con Intro. Los comandos con argumento [val] **muestran el valor actual si se invocan sin argumento**. Hay dos regímenes de guardado — el firmware siempre dice en su respuesta cuál acaba de tocar:

- Las preferencias se guardan solas e imprimen [auto-saved: ...]: TZ, TO, LT, WU, SPL, SV, PO, AO — más CT (guarda él solo el grupo PID que deriva) y LC (se guarda solo con PASS).
- Los parámetros del lazo viven en RAM hasta que los guarde — todo lo que edita el sintonizador: ganancias, parámetros LTIC/Lars/algo-12, LA, SP. La respuesta nombra el grupo exacto para conservar el cambio ([not saved – run 'ES ...']).

Versión, ayuda, estado

Comando	Qué hace
V	versión, autores, créditos
H / ?	lista de comandos · H TZ = detalles de zonas horarias
SW	marcas de agua de pila, heap libre, uptime + su origen, ppm del MCU contra el GPS

Salida (el DAC)

Comando	Rango	Qué hace
SP [n]	1..65535 (sin argumento = 32767, punto medio ≈1,65 V)	fija el DAC de control directamente — mando manual para experimentos
up1/up10/dp1/dp10	—	empujones de PWM ±1/±10 (rechazados durante una calibración)
DAC	—	informe: ruta de salida, códigos de 24 y 16 bits, Vctl medido, un paso en μHz (requiere CT)

Calibración

Comando	Duración	Qué hace
C	~2 min	el centrado PWM de dos puntos, más antiguo
CT	~3 min	sensibilidad K del oscilador + autoajuste de PID 3–9 y LTIC; guarda el PID él solo — ejecútelo primero
LC	~5 min	calibración del detector de fase (ns/V, offset, rango); se guarda solo con PASS — ejecútelo segundo
ACG g [cap]	—	accionamiento del centrado de ACQ: ganancia 50..20000 LSB/V, paso máx. 5..1000 LSB
AP	—	armar el divisor picDIV a mano

Modo e informes

Comando	Qué hace
RH / RD	informe legible / con tabuladores
RP / RR	pausar / reanudar el informe de 1 Hz
MH / MD	holdover (congelar el PWM, volar solo) / disciplinado
F	purgar los búferes de anillo del promediado de frecuencia
T [baud]	túnel GPS transparente por USB para u-center, 300 s (baud 4800..921600)

Selección de algoritmo y PID clásico (véase la Parte 4)

Comando	Rango	Qué hace
LA [n]	0..12	elegir / mostrar el algoritmo del lazo
LP [n]	algo 0..9	listar parámetros PID
KP/KI/KD n val	algo 3..7, 0..100000	fijar ganancias
IL n val	algo 3..9, 100..100000	pinza del integrador
BC / BS	0,0001..1,0 Hz	cruce / escala de mezcla del algo 8
NS	1..10000 LSB	paso máximo del algo 9

LTIC (algo 10) — guardar con ES LTIC, listar con LL

Comando	Rango / omisión	Significado
LNV	0..1e6	pendiente del detector, ns por voltio (la mide LC)
LZO	0..3,3 V	offset de cero del detector, en voltios
LRN	0..1e9	rango del detector, ns — <i>nota: este comando fija el rango del detector; el significado «autoaprendizaje on/off» del help es inalcanzable en v1.06 (véanse las notas)</i>
LAT	0,001..1e9 ns (100)	umbral de fase de ACQ
LDT	1e-13..1,0 (5e-10)	umbral de deriva DPLL→LOCK
LIV	1..600 s (300)	intervalo de corrección en LOCK
AQP/AQI/AQD/AQL, DPP/DPI/DPD/DPL, LKP/LKI/LKD/LKL	0..100000	PID por etapa
LPOL	-1 / 0 / +1	polaridad PWM→fase (0 = auto)
LCV	0..3,3 V	objetivo de centrado de ACQ
FA/FAD/FAL	10 / 100 / 1000 s	ventana de media del término amortiguador (ambas / DPLL / LOCK)

LTIC-Lars (algo 11) — guardar con ES LTIC

LG (0..10000, 0=auto), LD (por defecto 3), LTC (1..600 s, por defecto 60), LFD (1..100, por defecto 2), LTO (0..4095 ADC, por defecto 2620), LPL (1..10000 ns, por defecto 100), LPF (1..100, por defecto 5), LTK (± 32000 , 0=apagado), LTR (0..4095) — significados en la Parte 4.4.

Algo 12 (acumulador multinivel) — guardar con ES ALG012, listar con ML

MG (0..10000 LSB/ns, 0=auto desde CT), MR (0..10, por defecto 9), MF (0..3 origen de límites, por defecto 0), MFT (0=3600 s, o 2048..65535 s), MLP <nivel> <unidades> (nivel 0..10, valor 1..500000) — significados en la Parte 4.5.

GPS, hora, sensores

Comando	Rango	Qué hace
SV 0\ 1	—	survey-in / Time Mode apagado/encendido (aplica al próximo arranque) — se guarda solo

Comando	Rango	Qué hace
TZ <ciudad\ reg la>	p. ej. TZ Adelaide	zona horaria con DST (H TZ para detalles) — se guarda sola
TO <h[:mm]\ A>	-14..+14	desfase UTC fijo, o A = automático desde la posición GPS (regla DST de la UE) — se guarda solo
LT 0\ 1	—	mostrar UTC / hora local — se guarda solo
PO <f>	-5000..5000 Pa	offset de presión añadido a la lectura del BMP280 — se guarda solo
AO <f>	-3000..3000 m	offset de altitud añadido a la altitud del GPS — se guarda solo
SAW 0\ 1	—	corrección de diente de sierra (qErr) apagada/encendida — SAW a secas muestra el estado; recomendado ON con receptor de tiempo (guardar: ES FLAGS)
WU 0\ 1	—	calentamiento del OCXO al arrancar — se guarda solo
SPL 0\ 1	—	animación de arranque apagada/encendida — se guarda sola

Guardar, recordar, reiniciar

Comando	Qué hace
ES [obj]	guardar todo (sin argumento) o un grupo: TZ, PID, LTIC, FLAGS, ALG012, ALGO, PO
ER	recordar — releer los ajustes del flash ahora (deshacer cambios sin guardar)
EE	borrar el hueco de ajustes — omisiones en el próximo arranque
EW	desgaste del flash ring: ciclos de borrado, huecos usados, sector/dirección
FR	estado del anillo de solo lectura (siempre activo desde v0.96)
CS	estadística de correcciones: recuento, pico, RMS de las últimas 100/1k/10k/100k correcciones — pequeño y estable es bueno, creciente es malo
RB	reinicio en caliente (conserva ajustes — pero no los guarda antes él solo)
CR YES	reinicio en frío: borra ajustes + datos aprendidos, omisiones de fábrica (el YES es obligatorio, porque el modelo aprendido tarda días en reconstruirse)

Peculiaridades conocidas de v1.06 (lista honesta)

- MZ** aparece en el help pero no tiene manejador — la cancelación en el cruce por cero está siempre activa en el código; no hay nada que conmutar.
- LRN** queda eclipsado: la tabla de flotantes de LTIC lo reclama primero, así que **LRN** fija el *rango del detector en ns*. El conmutador de autoaprendizaje **LRN 0|1|R** del help es inalcanzable en esta compilación.
- FR 0|1** del help del firmware está caducado — el anillo está siempre activo; **FR** ignora los argumentos.
- H CS**, mencionado por la salida de **CS**, no existe como subpágina; **H** a secas imprime la lista completa.

Parte 8 — El sintonizador en el PC

`tools/gpsdo_tuner.py` — una consola de escritorio para observar y ajustar. Hace todo lo que hace un terminal, más gráficas en vivo y registro CSV.

8.1 Instalación y arranque

Python 3.9 o más nuevo, luego:

```
pip install PySide6 pyqtgraph pyserial tzdata
```

(`tzdata` solo lo usa el botón *Generate tz_table.h*; en Linux/macOS el sistema ya lo trae.)

Ejecutar: `python gpsdo_tuner.py` (en Windows también con doble clic).

8.2 Conexión

Elija el puerto en la barra, baud 115200, Connect. El sintonizador lee de inmediato la versión del firmware y **todos** los parámetros actuales (`LL`, `FA`, `LP 3–LP 9`, todos los verbos de Lars y del algo-12, `ML`) y rellena cada pestaña — ve el estado real del aparato, no las omisiones. La línea de estado muestra `state: LOCK (locked)` etc.; los títulos de las gráficas siguen al algoritmo activo.

8.3 Las pestañas

Pestaña	Qué edita
LTIC (algo 10)	los tres cuartetos de PID por etapa; Read (<code>LL</code>) / Save (<code>ES LTIC</code>) / Revert (<code>ER</code>)
LTIC-Lars (algo 11)	<code>LG LD LTC LFD LTO LPL LPF LTK LTR</code> con botones Set
Multi-level (algo 12)	<code>MG MR MF MFT</code> + la tabla completa de límites de 11 filas (<code>Send limits</code> , <code>Read all</code> , <code>List (ML)</code> , <code>Save (ES ALG012)</code>)
FA damping	ventanas de amortiguación <code>DPLL / LOCK</code>
PID algo 3-9	<code>Kp/Ki/Kd/IL</code> por algoritmo
Calibration	<code>LNV LZO LRN LCV LAT LIV LPOL</code>
Raw monitor	todo lo que dice la placa, sin analizar; controles de registro
Help	la referencia de comandos del firmware

Recuerde que la regla del firmware también aplica aquí: los botones **Set** solo cambian RAM; el botón **Save** de la pestaña envía el `ES ...` que persiste.

8.4 Las gráficas

Tres paneles, refrescados sin parar desde la telemetría de 1 Hz. Los paneles superiores dependen de la familia del algoritmo — fase (`dph/ph`) + tensión del detector para 10/11/12 (con el ancla gris y los bordes rojos de la banda válida, cuando la calibración se conoce), deriva + tensión de control para los clásicos. El panel inferior es siempre el error de frecuencia en Hz. Barra: selector de intervalo (1 min ... all), Follow live, Clear. La resolución es el ritmo de la telemetría — lo más rápido que ~2 s es invisible, y las gráficas confían en la placa.

8.5 Registro (las gráficas no son un registrador)

Raw monitor → **Start logging**, formato Full log / CSV only / Both:

- Full log** `gpsdo_YYYY-MM-DD_HH-MM-SS.log` — cada línea tal cual, ~217 MB por semana a 1 Hz.
- CSV** — columnas analizadas: `utc`, `up_s`, `algo`, `state`, `dph_ns`, `qerr_ns`, `vphase_v`, `pwm`, `f10`, `f100`, `ph_ns`, `level`, `corr`, `sig_ns`, `zc`, `bmp_c`, `sat`, `hdop` (~65 MB/semana). Celda vacía = el campo no estaba ese segundo (nunca cero).

- **Redact position** (ON por defecto) limpia Lat/Lon/Alt del registro completo guardado — comparte registros sin publicar su tejado.

Parte 9 — Ajustes, el flash ring y el desgaste

Todo lo persistente vive en un **anillo con nivelación de desgaste** en el sector 7 del flash: 255 huecos de 512 bytes. Cada guardado (un **ES**, un **LC** aprobado, una actualización de datos aprendidos) escribe el siguiente hueco en blanco; el sector se borra solo cuando el anillo da la vuelta — una vez cada 255 guardados. Al ritmo de este firmware (incluso un banco ocupado hace ~73 guardados al día) el borrado cae cada ~3,5 días, y el flash del F411 soporta ~10 000 ciclos por sector: **unos 96 años**. No lo desgastará. **EW** muestra los contadores reales cuando dé curiosidad.

Dos clases de registro comparten el anillo:

- **Ajustes** (**ES** y compañía) — todo lo que usted eligió, guardado solo cuando usted lo pide. Los guardados parciales (**ES** **PID** etc.) escriben *todo* el bloque de ajustes sembrado desde el último almacenado — así guardar un grupo no puede pisar otro.
- **Datos live** — lo que el aparato aprende solo: la calibración **LC**, el modelo de deriva/amortiguación, el último punto de trabajo. Se guardan automáticamente, pero con histéresis (solo cuando algo cambió de verdad en una magnitud sensata, y como muy pronto cada 20 minutos).

Actualizaciones de firmware: los ajustes de v1.04/v1.05 se migran al recordar (v4→v5 añade el bloque algo-12 con omisiones); un bloque de una versión radicalmente distinta, o un hueco corrupto, se trata como «no está» — omisiones, y el firmware pide calibración en el próximo arranque. El propio anillo se auto-repara: un hueco escrito a medias por un corte de corriente falla el CRC y simplemente se salta; gana el buen anterior.

La lista práctica, una última vez:

- regrabado rutinario: seguro, los ajustes sobreviven (Parte 2),
- tras una sesión de ajuste: **ES**,
- antes de vender/regalar una unidad: **CR YES**,
- cuando la unidad se porte raro tras una sesión de experimentos: **ER** (recuperar los ajustes guardados y conocidos-buenos), y solo después diagnosticar.

Parte 10 — Resolución de problemas

Síntoma	Causa probable → solución
No compila, error 1toa	núcleo STM32 3.0.0 — baje a la 2.12.0 (Parte 1.1)
Compila, imprime ? o basura donde van números	Tools → C Runtime Library → Newlib Nano + Float printf/scanf
TFT blanca tras compilar con el núcleo 3.0.0	lo mismo — núcleo 2.12.0
TFT blanca con núcleo 2.x	driver equivocado en User_Setup.h o falta TFT_MISO PA6 (Parte 1.4)
El arranque se cuelga justo tras TFT: init start	igual que arriba — revise cableado y el #define del driver
Falta el serie USB tras grabar	pulse RESET; si sigue — driver Zadig para VID 0483/PID 5740 (Parte 2.5)
Ajustes/calibración desaparecidos tras grabar	alguien usó Erase Chip — es la regla del sector 7 (Parte 2.2); repita CT, LC, ES
LED amarillo apagado	sin fix GPS — antena, cielo, cable
HDOP:TIME nunca aparece	survey-in sin terminar (requiere receptor de tiempo + sv 1 + buen cielo); se repite tras cada corte de alimentación
CT falla / rechaza K	cableado EFC o sensibilidad del oscilador fuera de 0,02–2 mHz/LSB — revise el búfer del DAC y el rango EFC
LC falla	ejecute CT primero; no mueva la unidad durante los 5 minutos
Tendencia clavada en NoCT (algo 12)	ganancia en auto pero CT nunca corrió → CT
Tendencia clavada en ACQ / NoPL, algoritmos 10–12	LPOL sin fijar (el lazo imprime el aviso y espera) — ponga LPOL -1 o 1, luego ES LTIC
El algo 12 corrige sin parar en el nivel 0	sitio ruidoso — MF 1 (la tabla de Alan), véase la Parte 4.5
La fase se va de rampa tras SAW 1	no hay receptor de tiempo / qErr no válido — SAW 0
Las líneas del informe se congelan cuando un programa abre el CDC y no lee	arreglado en v1.06 (escrituras no bloqueantes + cola de 1 KB); si lo ve, está en firmware antiguo
LED azul parpadeando rápido	fallo de firmware capturado (aserción / pila / heap) — lea el mensaje del serie; repórtelo
La unidad se reinicia sola	revise el carril 3V3 bajo la carga del OCXO; el rótulo de arranque imprime la causa del reset
Pantalla viva, sin salida serie	¿está solo en Bluetooth? 57600 en Serial2; o alguien escribió RP — RR reanuda

Si nada de eso ayuda: capture un registro completo con el sintonizador (Parte 8.5), anote la versión de v y pregunte — con el registro adjunto. Una semana de CSV a 1 Hz es la diferencia entre adivinar y saber.

Apéndice A — Cómo funciona un GPSDO, en palabras llanas

Puede usar este aparato sin leer este apéndice — pero el día en que algo se porte raro, estos veinte párrafos le dirán *por qué*.

A.1 El problema

Quiere una señal de 10 MHz que esté bien **ahora** (segundo a segundo) y bien **para siempre** (a lo largo de meses y años). Dos piezas aportan media mitad cada una, y ninguna las dos:

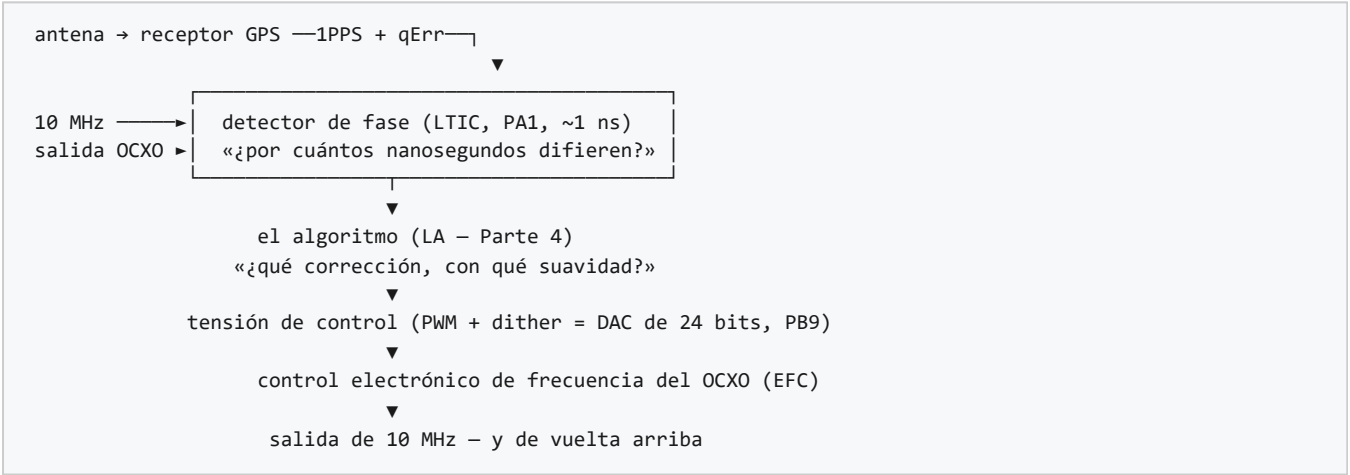
	Corto plazo (segundos)	Largo plazo (meses)
Solo el OCXO	excelente — uno bueno se mantiene en partes de 10^{11}	deriva — el envejecimiento y la temperatura lo desvían despacio
Solo el 1PPS del GPS	ruidoso — cada pulso cae a $\sim \pm 25$ ns de la verdad	excelente — gobernado por relojes atómicos, para siempre

La parte de «horno» del OCXO ya es media historia: el cristal vive en una cajita calentada a una temperatura constante (ese es el calentamiento de 300 segundos al arrancar), porque la temperatura es su mayor enemigo. Lo que queda es el envejecimiento — un paseo lento que nadie puede apagar.

El pulso del GPS es lo contrario: cada segundo individual es solo aproximado (el receptor cuantiza el tiempo, la señal se retrasa en la ionosfera, rebota en los edificios), pero el *promedio* de horas está clavado al tiempo atómico, porque los satélites cargan relojes atómicos y sin eso el GPS no funciona.

Un GPSDO es un reparto de trabajo: el OCXO se ocupa de los segundos, el GPS de los meses, y un lazo de control entre ambos mueve al primero, en pasitos diminutos, manteniéndolo anclado al segundo.

A.2 El lazo, pieza a pieza — y dónde está cada una en esta placa



- El **detector de fase** responde «¿por cuánto nos retrasamos?» una vez por segundo, en nanosegundos. Ese único número — la *fase* — es el marcador en el que se juega todo el partido.
- El **contador (TIM2)** responde «¿vamos rápidos o lentos?» contando literalmente los 10 MHz. Es grueso (pasos de 0,01 Hz) pero directo.
- El **algoritmo** mira esos números y decide una sola cosa: cuánto empujar la tensión de control, y en cuánto tiempo repartir el empujón.
- El **DAC**: PWM de 65536 pasos, con dither hasta unos efectivos 24 bits (pasos de $\sim 0,2 \mu V$ — el oscilador nota menos de un paso PWM, así que el firmware recuerda la fracción entre pasos).
- El **pin EFC del OCXO**: voltios dentro, hercios fuera — todo el rango de ajuste mide apenas unos pocos hercios de ancho, y por eso los microvoltios importan.

A.3 Fase, frecuencia y la única multiplicación que hace falta

El error de frecuencia y la fuga de fase son el mismo hecho en dos disfraces. Si el oscilador se desvía en una fracción, su fase se desliza exactamente a ese ritmo:

Error de frecuencia a 10 MHz	La fase se desliza
1 Hz	100 ns por segundo
0,01 Hz	1 ns por segundo
0,001 Hz (1 mHz)	1 ns cada 10 segundos

Dos consecuencias dignas de grabar:

1. Si la fase se detiene, las frecuencias son iguales — sea cual sea el valor de la fase. La meta real del lazo es una fase *quieta*, aparcada cerca de cero.
2. Los errores que vale la pena perseguir son absurdamente pequeños. Un milihercio a 10 MHz es una parte en 10^{10} — y el lazo distingue rutinariamente diez veces mejor. Por eso cada cable, cada microvoltio y cada grado importan aquí más que en cualquier otro circuito que haya construido.

A.4 Por qué las correcciones deben ser suaves (el corazón del asunto)

Suponga que el pulso del GPS llega 20 ns tarde porque la señal rebotó en un edificio. El «error» de fase medido es 20 ns — pero no es real: el oscilador no se movió. Si el lazo corrige al instante a valor nominal, **copia el ruido del GPS sobre el oscilador** y la salida acaba *peor* que el OCXO libre. Ese único error es lo que separa un GPSDO de un oscilador apaleado por el GPS.

La defensa es promediar: el ruido aleatorio se reduce con $\sqrt{N}$. Promedie 100 segundos $\rightarrow$ ruido $\div 10$; 1000 s $\rightarrow \div 31$. El lazo espera, pues, a que los números promediados del GPS queden más silenciosos que la propia deriva del oscilador, y solo entonces actúa — con una corrección a la medida de lo que el promedio largo realmente demostró, repartida en un tiempo comparable. Por eso:

- el algoritmo 11 tiene LTC (una constante de tiempo — «cuánta paciencia tiene este lazo»),
- el algoritmo 12 deja que el tamaño del error elija su propio tiempo de promediación (un error grande queda demostrado en segundos y se corrige en segundos; uno diminuto, a lo largo de una hora y a lo largo de una hora),
- cada corrección de este firmware se reparte en pasos sub-LSB en vez de volcarse de golpe.

Un almuerzo gratis: la **corrección de diente de sierra (SAW 1)**. El receptor *sabe* cuánto cuantizó cada pulso (el número `qErr`) y lo confiesa cada segundo. Restar esa confesión convierte ± 25 ns de error falso en unos pocos ns antes de que empiece promedio alguno.

A.5 Holdover

Cuando el GPS desaparece (antena cortada, receptor muerto), al lazo le falta la verdad con la que gobernar. La jugada correcta es **congelar** la última tensión de control buena y dejar que el OCXO navegue con su propia estabilidad — eso es el holdover (MH manual, o automático al perder fix; el LED amarillo parpadea, la pantalla se pone naranja, `[HOLDOVER]` sustituye a la tendencia en el informe). El oscilador derivará entonces con el envejecimiento y la temperatura — despacio, pero imparable, hasta que el GPS vuelva.

A.6 Cómo se juzgan los resultados: la desviación de Allan en un párrafo

Los patrones de frecuencia se comparan con la **desviación de Allan (ADEV)**: más o menos «cuánto discrepa el promedio consigo mismo, tomado sobre τ segundos», graficada contra τ . La curva de un GPSDO cae de izquierda a derecha: en $\tau = 1$ s se ve el ruido propio del oscilador; al crecer τ , el promedio vence al ruido y el anclaje del GPS toma el mando (en el banco del autor: $\sim 3 \cdot 10^{-9}$ a 1 s cayendo hasta $\sim 5 \cdot 10^{-12}$ a 3000 s). Cuando alguien publica « $1e-12$ en tau 10 000», esa es la frase que dice. Un *joroba* en medio de la curva significa que el lazo pelea consigo mismo — demasiado rápido para su propio ruido — y es la firma clásica de un GPSDO mal ajustado.

Apéndice B — PID para reacios

Nunca *necesita* este apéndice para usar el aparato — CT ajusta los lazos por usted. Es para el día en que abra la pestaña PID algo 3-9 del sintonizador, o escriba KP, y quiera saber qué palanca sostiene y hacia dónde muerde. No se usa matemática más allá de la multiplicación.

B.1 La imagen de la ducha

Todo lazo de control jamás construido hace las mismas cuatro cosas:

- 1. Medir dónde está (la temperatura del agua).
- 2. Comparar con dónde debería estar (lo cómodo).
- 3. La diferencia es el error (frío por 5 grados).
- 4. Actuar (abrir el grifo del caliente), esperar, repetir.

La única pregunta de toda la materia es el paso 4: *¿cuánto abrir el grifo?* PID — tres letras, tres respuestas — es la receta estándar.

B.2 P — proporcional: «reacciona al ahora»

Abra el grifo en proporción al error. Frío por mucho → abra mucho. Frío por poco → abra poco.

- **P a solas tiene un defecto:** necesita algo de error para producir acción alguna, así que se asienta *cerca* del objetivo, nunca en él — el último grado de frío no alcanza para mantener el grifo abierto. Ese residuo se llama **droop**.
- **Demasiada P:** se pasa de caliente, luego corrige de más al frío, y oscila entre ambos — la ducha del infierno.
- **Muy poca P:** tarda una eternidad en llegar.

B.3 I — integral: «recuerda el pasado»

Mire el error *en el tiempo*. Aun un frío diminuto y persistente se acumula, en una suma corriente, hasta que el lazo añade acción suficiente para cerrarlo. I es lo que mata el droop — es el término «al final, exactamente».

- **Demasiada I:** el clásico yoyó lento. La suma crece durante la aproximación, luego se gasta como rebasamiento, se reconstruye al otro lado, y el lazo se mece con calma por una eternidad.
- **Windup:** si el grifo ya está abierto del todo (la salida en su límite), la suma sigue creciendo inútilmente y luego tarda siglos en deshacerse. El remedio es una pinza en la suma — y exactamente eso es IL (I_LIMIT).

B.4 D — derivativo: «anticipa»

Reaccione no al error sino a lo *rápido* que cambia. ¿Se caldea deprisa? Empiece a cerrar el grifo *antes* de llegar. D es el amortiguador — la suspensión que corta las oscilaciones que P e I adorarían.

- **El vicio de D:** amplifica el ruido de medición (derivar un sensor tembloroso produce agujas). Por eso los términos D suelen ir filtrados, pequeños, o ambas cosas — y por eso una medición de fase ruidosa empeora un ajuste cargado de D en vez de mejorarlo.
- En los algoritmos PLL (4/5/7) la ranura Kd actúa sobre la *fase* acumulada y no sobre una derivada cruda — el mismo oficio de amortiguar, con otro cableado. No deje que la letra despiste; piense «la perilla de la amortiguación».

B.5 Las tres letras, una tabla

Perilla	Reacciona a	Cura	Demasiada →	Muy poca →
P	el error de ahora mismo	la desgana	rebasamiento, oscilación rápida	eternidad en llegar
I	el error acumulado en el tiempo	el offset permanente (droop)	yoyó lento, windup	se asienta cerca, nunca en el objetivo

Perilla	Reacciona a	Cura	Demasiada →	Muy poca →
D	la velocidad de cambio del error	el rebasamiento (amortiguación)	nervios persiguiendo ruido	el rebasamiento resuena

B.6 Dónde están las perillas en este firmware

Lo que gira	Dónde	Qué letra es en realidad
KP n val / KI / KD / IL n val	algoritmos 3–9 (pestaña PID algo 3-9)	literalmente P, I, D y la pinza del integrador
AQP...AQL, DPP...DPL, LKP...LKL	las tres etapas del algoritmo 10 (pestaña LTIC)	un juego completo de PID por etapa — el lazo se reajusta solo mientras se asienta
LG / LD / LTC	algoritmo 11 (pestaña LTIC-Lars)	ganancia (con cuánta fuerza), amortiguación (cómo se asienta), constante de tiempo (con cuánta paciencia) — las mismas tres perillas con la ropa de Lars
MG y los límites	algoritmo 12 (pestaña Multi-level)	sin PID — véase abajo

El algoritmo 12 merece un párrafo honesto: **no tiene P, ni I, ni D**. En vez de ganancias fijas pregunta, cada vez, «¿qué tan grande es el error, cuánto promedié para demostrarlo?» y escala la corrección al par — los errores grandes reciben trato rápido y firme; los pequeños, trato lento y suave. Es la misma física con el ajuste incorporado, y por eso sus únicas perillas manuales son la ganancia (MG, y CT la mide por usted) y los límites que deciden qué cuenta como «grande».

B.7 Por qué existe CT — el problema de la regla

Los números del PID viven en unidades de «pasos PWM por hercio de error». Pero un paso PWM vale *una cantidad distinta de hercios en cada oscilador individual* — 0,32 mHz en uno de los Vectron del autor, casi siete veces menos en una construcción de EFC estrecho. Sin medir su oscilador, el mismo $k_p = 1000$ es un empujoncito suave en una placa y un empujón violento en otra. **CT mide la respuesta voltios-a-hercios de su oscilador y reescala cada ganancia a juego** — para que el ajuste de fábrica signifique lo mismo físico en cada unidad. Por eso el orden de calibración de este manual es ley: primero CT, después LC, el ajuste (si alguna vez) al final.

B.8 Diez reglas de pulgar

1. Ejecute CT y LC antes de tocar ganancia alguna. En la mayoría de las vidas, ahí termina el ajuste.
2. Cambie una perilla a la vez.
3. Duplique o parta por dos — nunca $\times 10$. El ajuste de lazos reacciona a proporciones, no a aritmética.
4. Dele una hora a cada cambio. Estos lazos promedian por minutos; juzgar un cambio a los treinta segundos es leer un libro por una letra.
5. Ante la duda, vaya **más lento** (más LTC, menos ganancia). Lento es solo aburrido; rápido es inestable.
6. Oscilación → recorte P primero, luego I.
7. Un offset que nunca termina de cerrar → más I — o, más probable, se saltó CT.
8. La salida persiguiendo visiblemente el ruido del GPS → lazo más lento, SAW 1, mejor cielo para la antena. No más amortiguación.
9. ES tras cada sesión; ER des-experimenta una mala tarde.
10. Si le pelea un día entero, sospeche del hardware antes que del ajuste: vista del cielo de la antena, temperatura (radiadores, puertas, sol), cableado EFC. A los lazos de este firmware es difícil romperlos y fácil culparlos.